

Burp Suite repeater request/response Analysis

NAME: Iresh Chirantha

ASSIGNMENT: Burp repeater request modification

Tools: Burp Suite, Web browser, OWASP juice shop

Testing Environment: Kali Linux

Purpose

This practical demonstrates the use of Burp Suite Repeater to resend captured HTTP requests, safely modify individual request parameters or headers, and compare the resulting server responses.

Scope

Testing was conducted against an authorized local OWASP Juice Shop laboratory environment. No external or unauthorized systems were tested.

Table of contents

Contents

1). Introduction.....	4
3). Objectives.	5
4). Methodology.	6
5). Test 1 – User-Agent modification.....	7
5.1). Original request.	7
5.2). Modified user-agent request and response.	8
5.3). Results	9
5.4). Observation.	10
6). Test 2 – Accept-language modification	11
6.1). Original request	11
.....	12

1). Introduction.

This practical demonstrates the use of Burp Suite Repeater for manually resending and analyzing HTTP requests. Burp Repeater allows a captured HTTP request to be modified and sent repeatedly so that the resulting server responses can be compared.

An authorized Metasploitable 2 virtual machine was used as the target environment. Metasploitable 2 is an intentionally vulnerable virtual machine designed for security testing and research. Its preinstalled web applications provide a suitable environment for practicing HTTP request analysis.

Three HTTP requests were captured using Burp Suite and transferred to Repeater. A single safe parameter or HTTP header was modified in each test, after which the original and modified responses were compared.

3). Objectives.

The objectives of this practical were to:

1. Configure communication between Kali Linux and Metasploitable 2.
2. Access a web application hosted on Metasploitable 2.
3. Capture HTTP requests using Burp Suite Proxy.
4. Send three captured requests to Burp Suite Repeater.
5. Modify one safe parameter or HTTP header in each request.
6. Resend the modified requests.
7. Compare the original and modified server responses.
8. Document the results using screenshots and comparison tables.

4). Methodology.

The practical was conducted in an isolated virtual laboratory containing Kali Linux and Metasploitable 2. Both virtual machines were connected through a Host-only network to prevent the intentionally vulnerable target from being unnecessarily exposed to external networks.

The IP address of Metasploitable 2 was identified and connectivity from Kali Linux was verified using ICMP ping. The web server hosted on Metasploitable 2 was then accessed through a web browser.

Burp Suite was used to intercept HTTP traffic generated while interacting with the web application. Three different HTTP requests were selected and sent to Burp Repeater. Each request was first sent without modification to establish a baseline response.

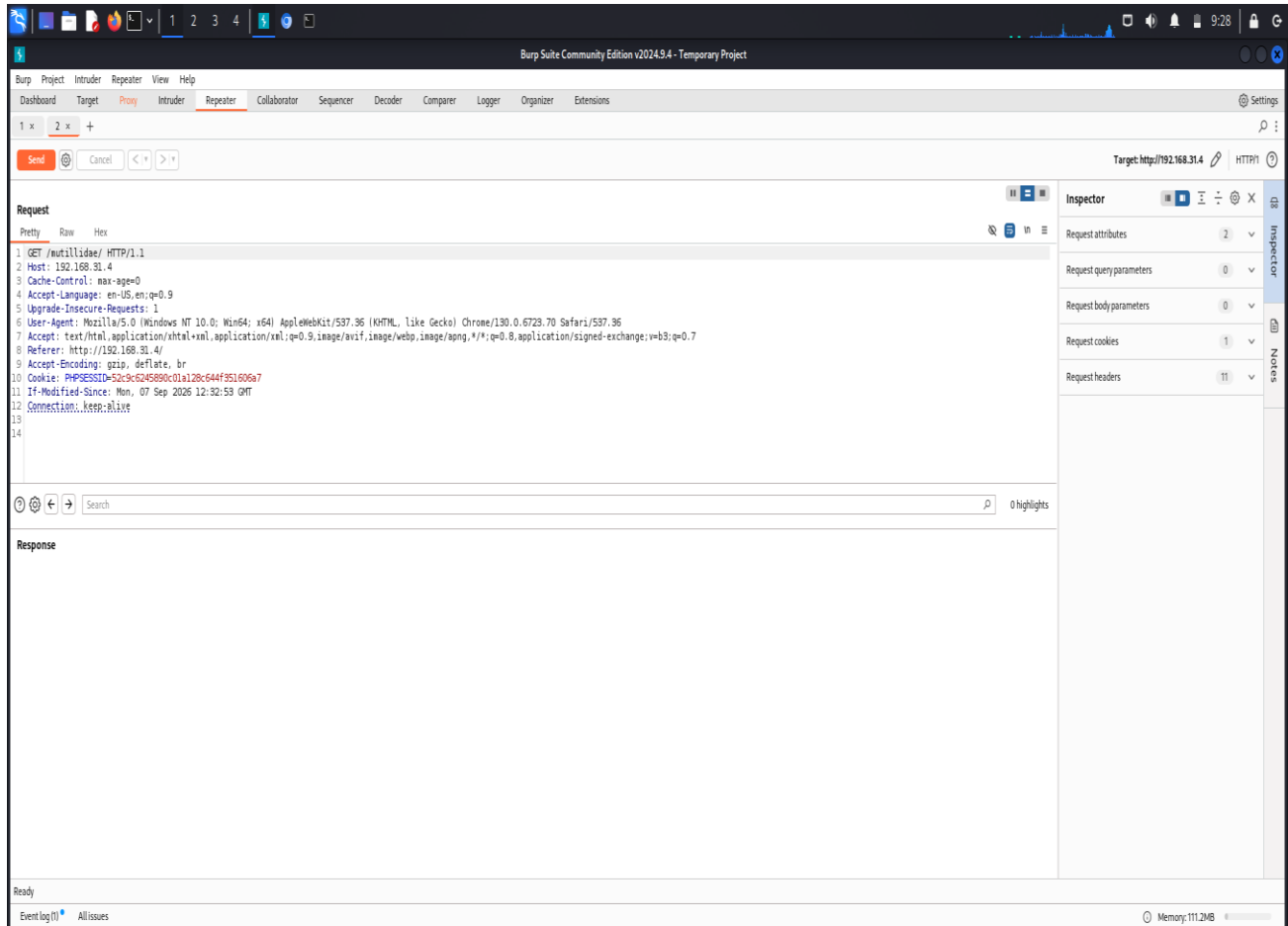
One safe parameter or HTTP header was then changed in each request. The modified request was sent again and its response was compared with the baseline. HTTP status codes, response length, response content, and other observable differences were recorded.

All testing was restricted to the authorized Metasploitable 2 laboratory environment and no destructive actions were performed.

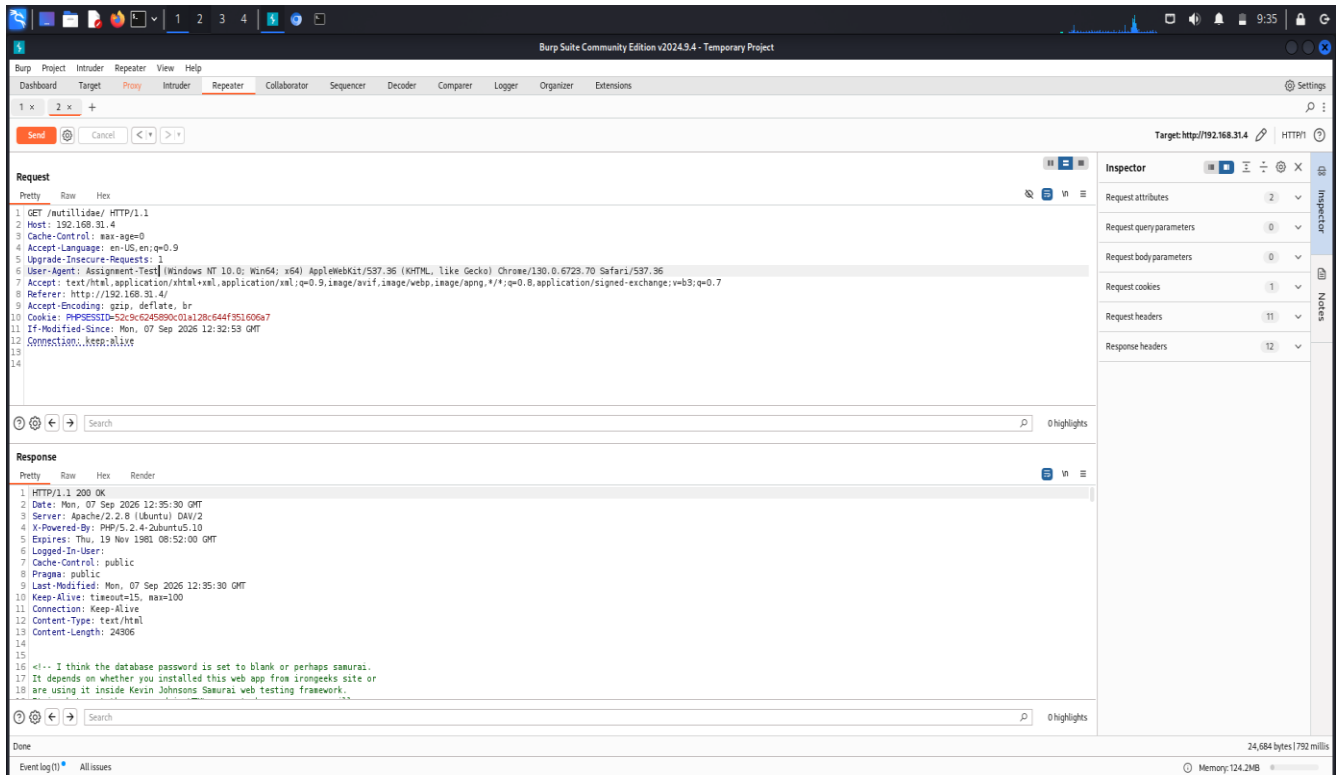
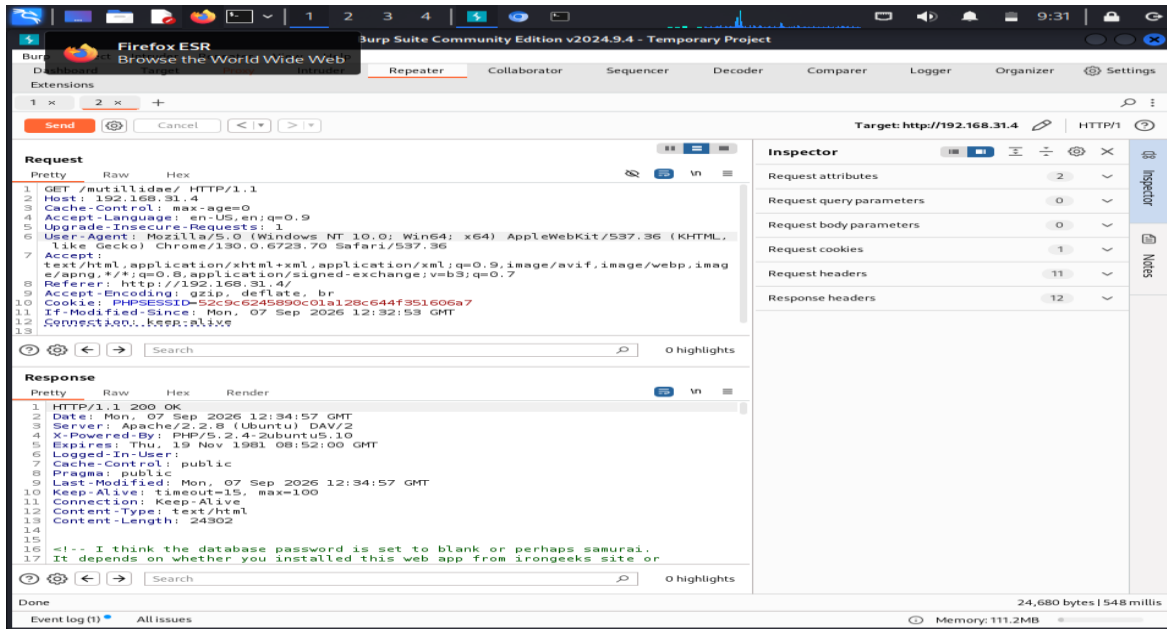
5). Test 1 – User-Agent modification.

5.1). Original request.

The first HTTP request was captured using Burp Suite Proxy and transferred to Burp Repeater. This was performed using the Metasploitable 2 web application.



5.2). Modified user-agent request and response.



The user-agent was changed from Mozilla/5.0 to Assignment-Test. No other section of the request was changed.

5.3). Results.

Attribute	Original	Modified
User-agent	Mozilla/5.0	Assignment-Test
HTTP status	200 OK	200 Ok
Response-length	24302	24306
Response Difference	Original webpage	Same webpage

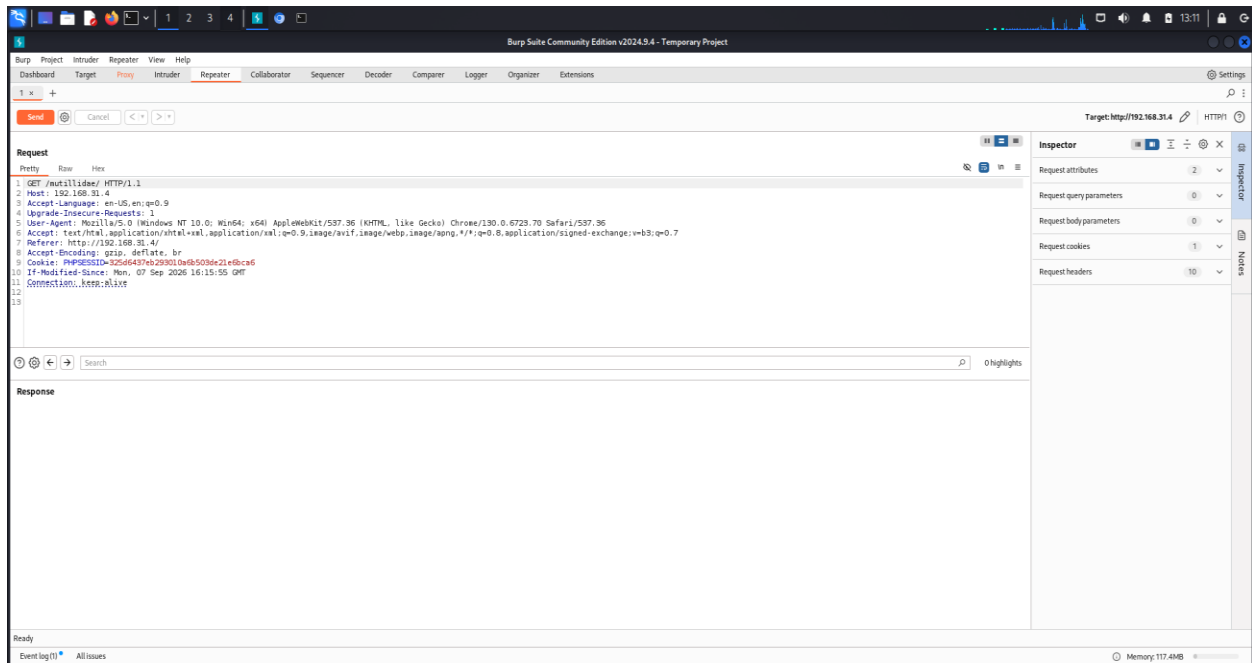
5.4). Observation.

The User-Agent header was changed from `Mozilla/5.0` to `Assignment-Test`. The server returned a `200 OK` response. The response content remained the same, so no significant difference was observed.

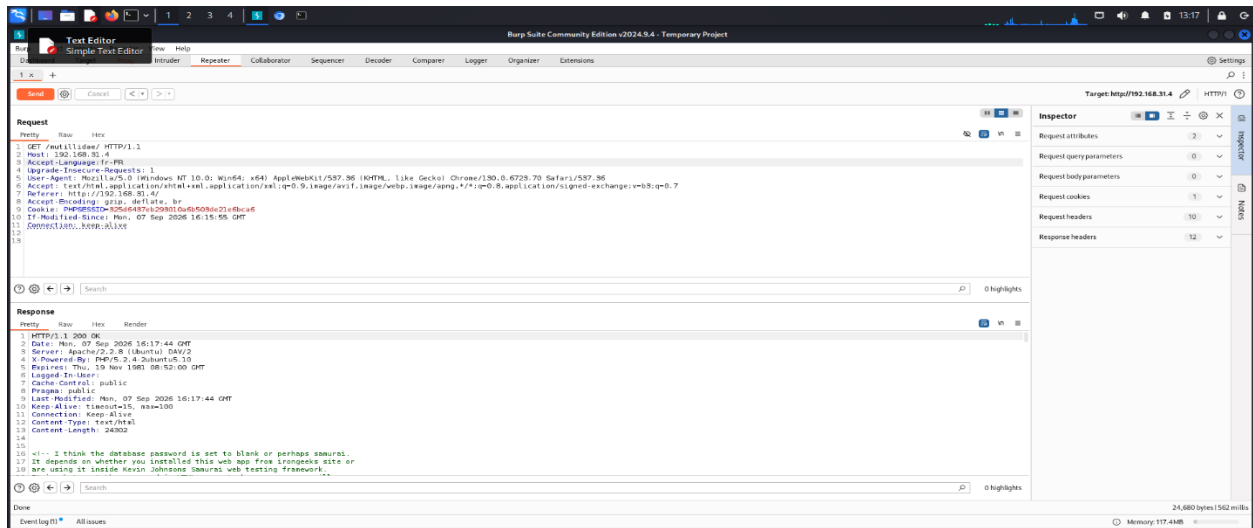
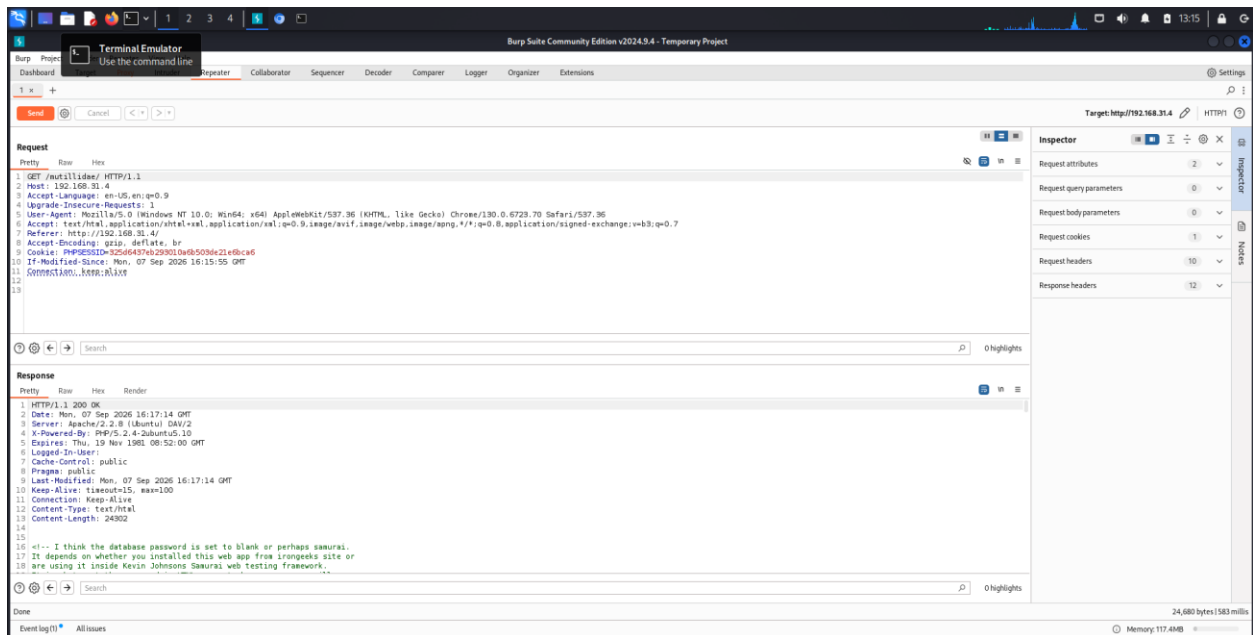
6). Test 2 – Accept-language modification.

6.1). Original request.

The second HTTP request was captured using Burp Suite Proxy and transferred to Burp Repeater.



6.2). Modification of Accept-language.



The Accept-Language header was modified from:

en-US,en;q=0.9

to:

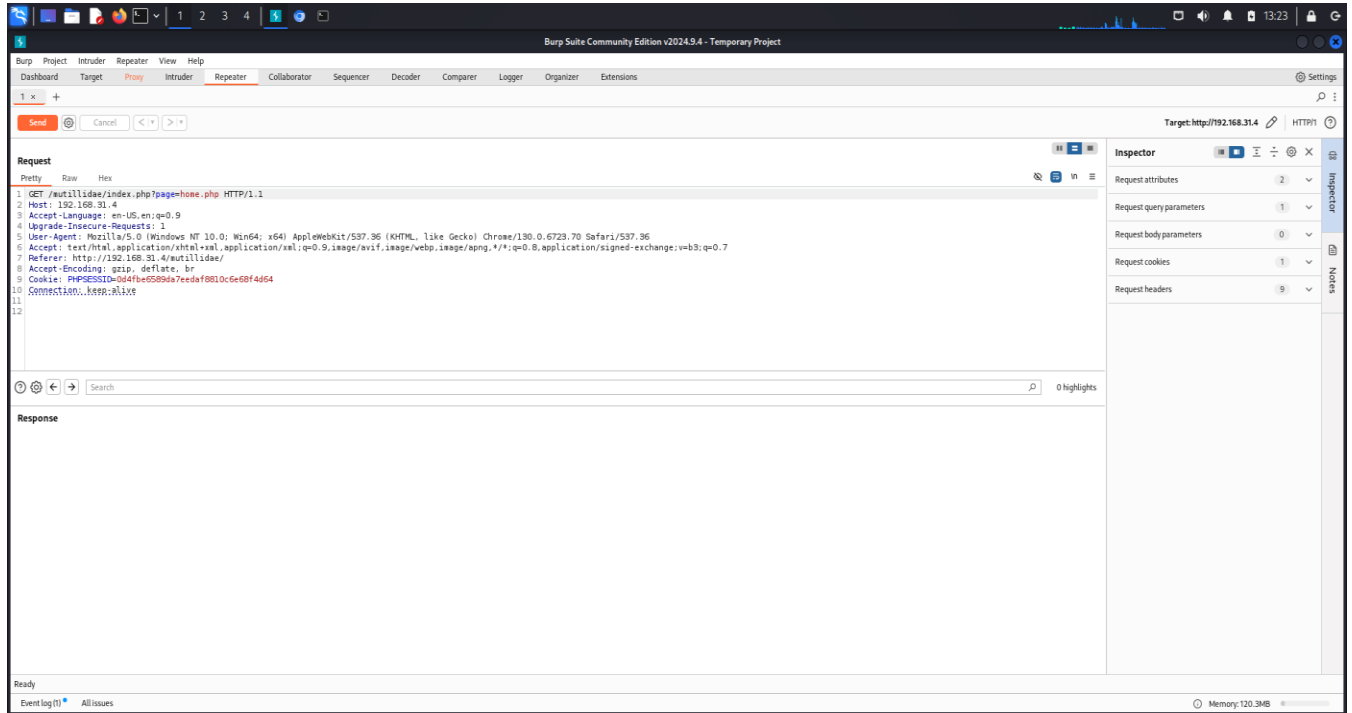
fr-FR

6.3). Results.

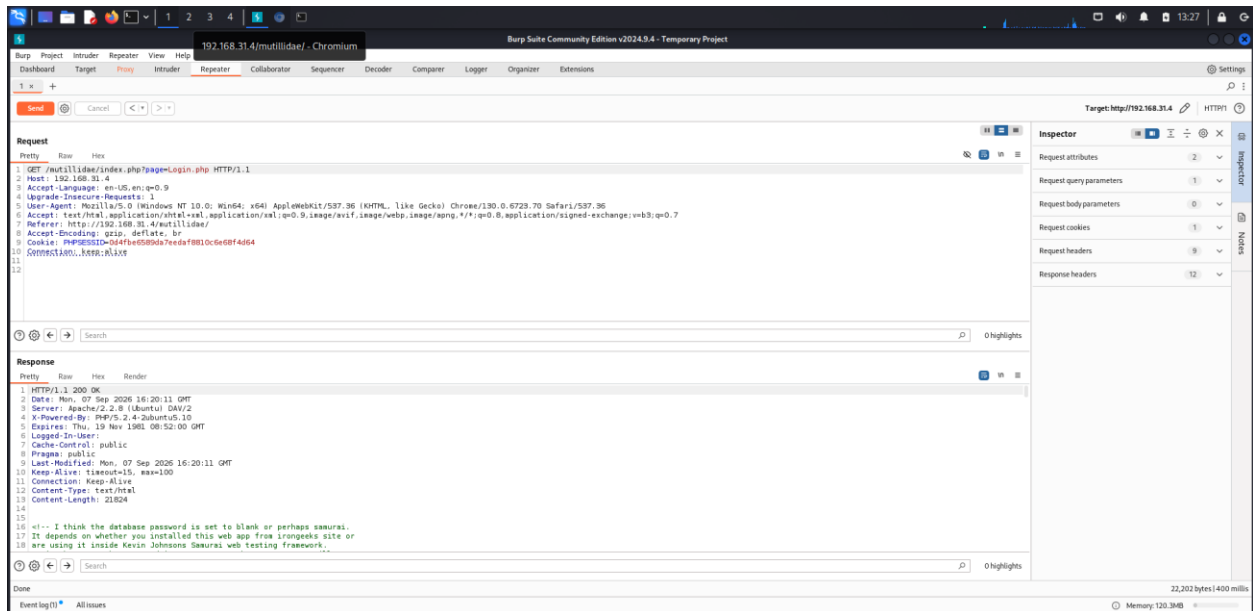
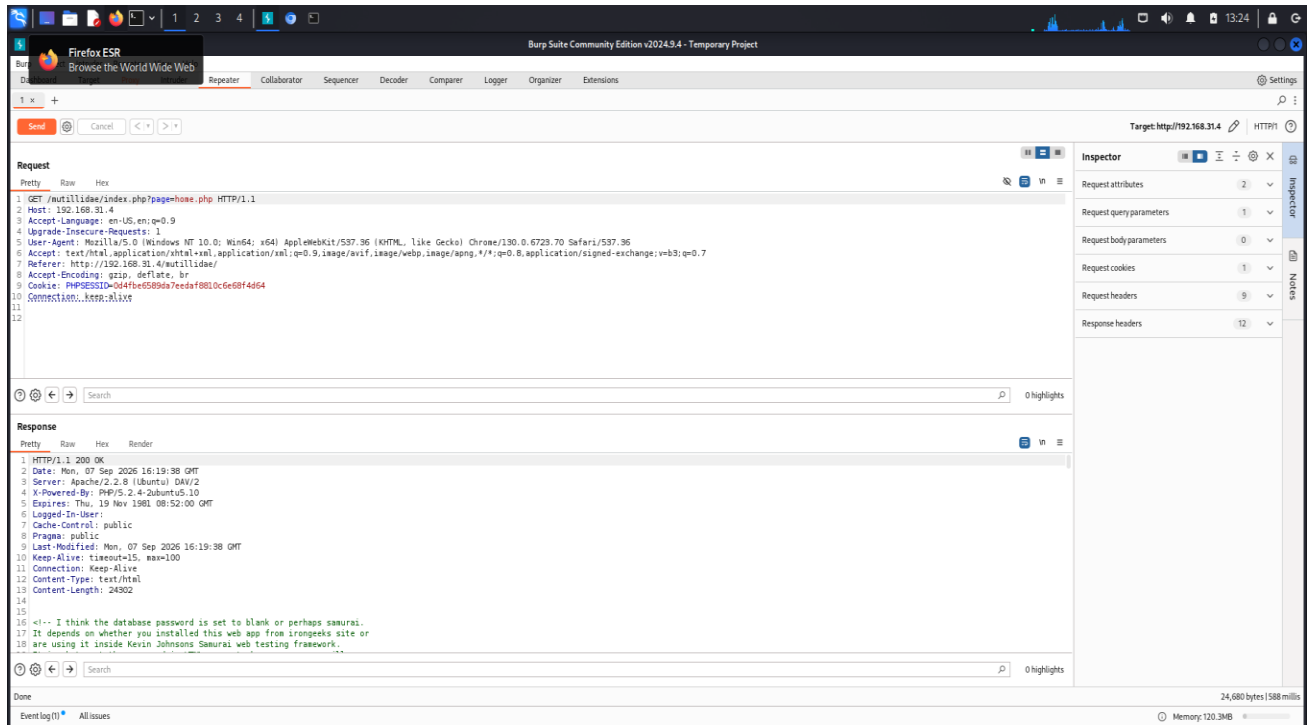
Attribute	Original	Modified
Accept-language	En-US,en;q=0.9	Fr-FR
HTTP status	200/ok	200/ok
Response length	24502	24502
Response difference	Same webpage	Same webpage

7). Test 3 – Parameter modification.

This test is almost identical, but this time you change **one normal URL** parameter.



7.1). Modification.



The third HTTP request was captured using Burp Suite Proxy and transferred to Burp Repeater.

7.2). Results.

Attribute	Original	Modified
Parameter	Page = home	Page = login
HTTP status	200 OK	200 OK
Response length	24502	21824
Response difference	Home page	Login page

7.3). Observation.

The `page` parameter was changed from `home` to `login`. The modified request returned a different webpage compared with the original request. This showed that the parameter affected the content returned by the application.

8). Conclusion.

The practical successfully demonstrated the basic use of Burp Suite Repeater in an authorized cybersecurity laboratory. Three HTTP requests were captured, transferred to Repeater, safely modified, and resent to the Metasploitable 2 web application.

The exercise provided practical experience in HTTP request analysis, request modification, and response comparison. It also demonstrated the importance of maintaining a baseline request and changing only one component at a time when analyzing application behaviour.